

Policy Gradients

CS 185/285

Instructor: Sergey Levine
UC Berkeley

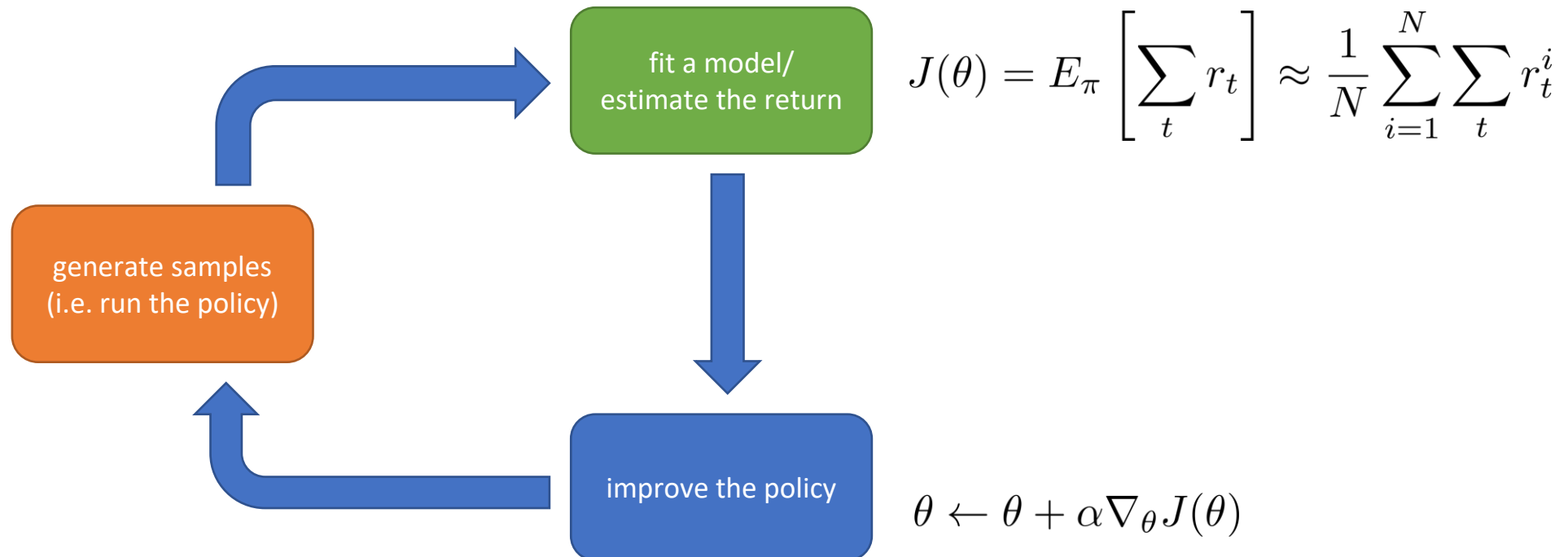


Part 1:

REINFORCE: basic policy gradients

Optimizing the objective of RL

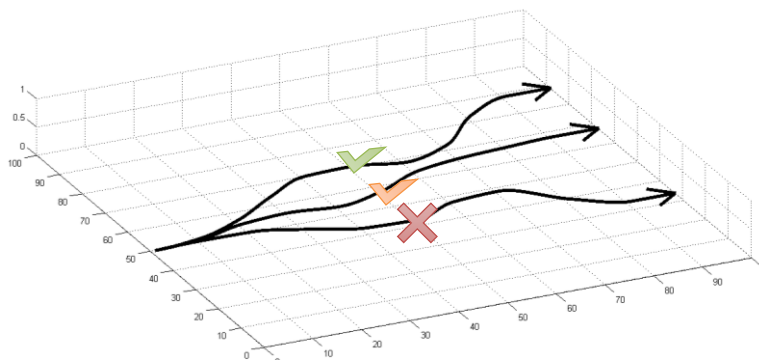
$$\theta^* = \arg \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right] \quad \underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_H, \mathbf{a}_H)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^H \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$



Evaluating the objective

$$\theta^* = \arg \max_{\theta} \underbrace{\mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right]}_{J(\theta)}$$

$$J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right] \approx \frac{1}{N} \sum_i \sum_t r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$$



所有t时刻的所有sample i

sum over samples from π_{θ}

Direct policy differentiation

$$\theta^* = \arg \max_{\theta} \underbrace{\mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right]}_{J(\theta)}$$

a convenient identity

$$\underline{p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)} = p_{\theta}(\tau) \frac{\nabla_{\theta} p_{\theta}(\tau)}{p_{\theta}(\tau)} = \underline{\nabla_{\theta} p_{\theta}(\tau)}$$

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\underbrace{r(\tau)}_{\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t)} \right] = \int p_{\theta}(\tau) r(\tau) d\tau$$

$$\nabla_{\theta} J(\theta) = \int \underline{\nabla_{\theta} p_{\theta}(\tau)} r(\tau) d\tau = \int \underline{p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)} r(\tau) d\tau = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) r(\tau)]$$

Direct policy differentiation

$$\begin{aligned}
 \theta^* &= \arg \max_{\theta} J(\theta) \\
 J(\theta) &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [r(\tau)] \\
 \nabla_{\theta} J(\theta) &= \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) r(\tau)]
 \end{aligned}$$

log of both sides

$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^H \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\log p_{\theta}(\tau) = \log p(\mathbf{s}_1) + \sum_{t=1}^H \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) + \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\nabla_{\theta} \left[\log p(\mathbf{s}_1) + \sum_{t=1}^H \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) + \log p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) \right]$$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$

Evaluating the policy gradient

$$\text{recall: } J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right] \approx \frac{1}{N} \sum_i \sum_t r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$

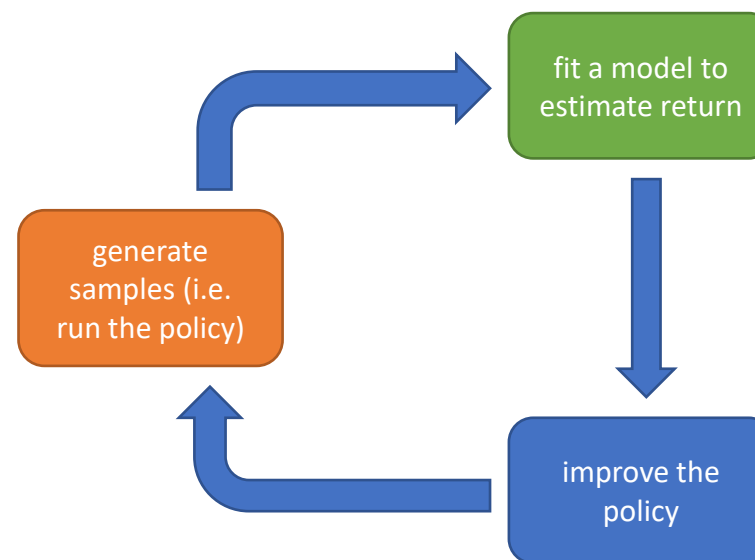
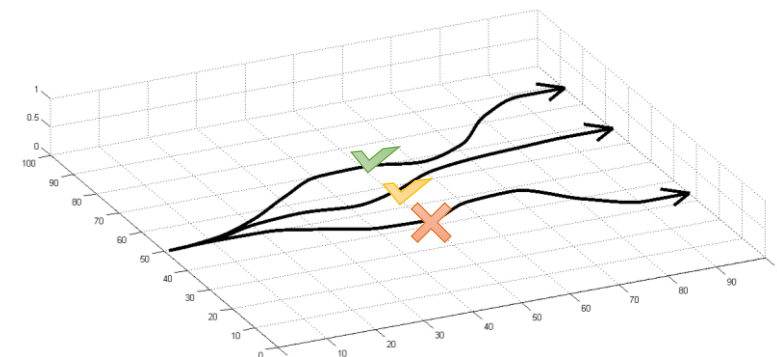
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right)$$

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

REINFORCE algorithm:

1. sample $\{\tau^{(i)}\}$ from $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ (run the policy)
2. $\nabla_{\theta} J(\theta) \approx \sum_i \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) \left(\sum_t r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right)$
3. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

利用总体目标函数 $J(\theta)$ (其实就是总体奖励函数) 的梯度, 来刻画其接下来的变化, 以此来优化策略, 每次优化后都必须重新sample环境



Part 2:

Understanding policy gradients

Evaluating the policy gradient

$$\text{recall: } J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right] \approx \frac{1}{N} \sum_i \sum_t r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})$$

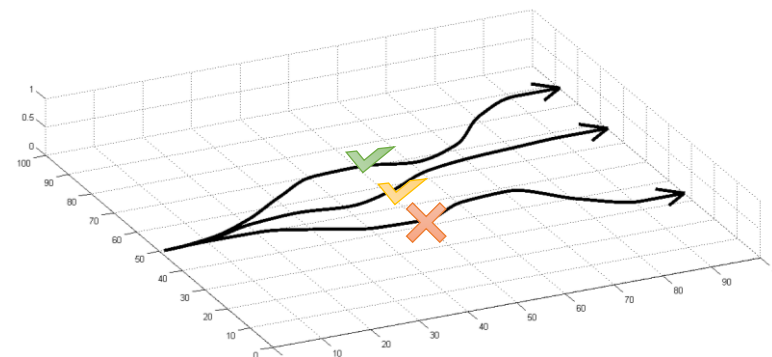
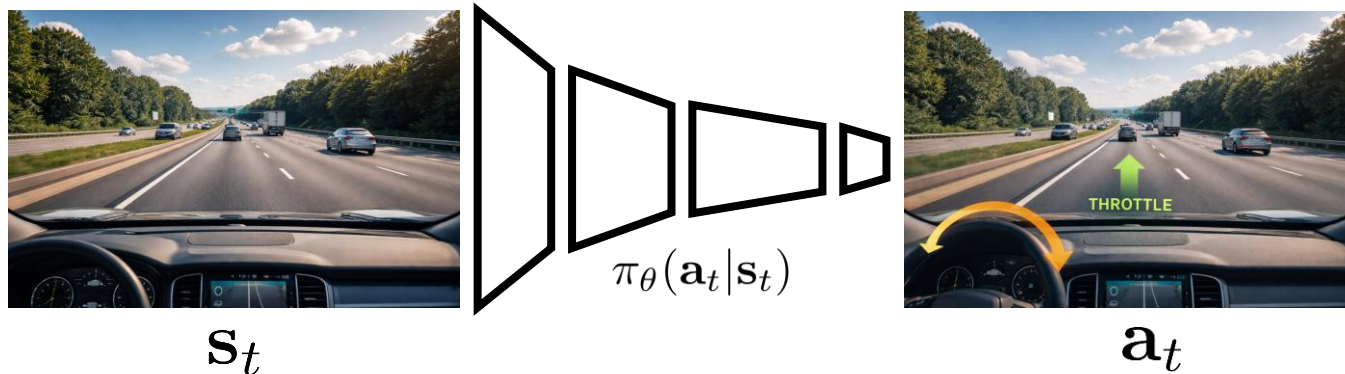
$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t) \right) \right]$$

策略梯度的精确表达式

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right)$$

由于在实际应用中，我们无法计算所有轨迹的期望，因此用蒙特卡洛方法进行近似

what is this?



可以直观看到，策略梯度会引导智能体向能获得更高累积奖励的策略方向优化，但如果没有优化手段，更新路径可能会比较曲折，这也是后续需要引入基线、Actor-Critic等优化方法的原因

Comparison to maximum likelihood

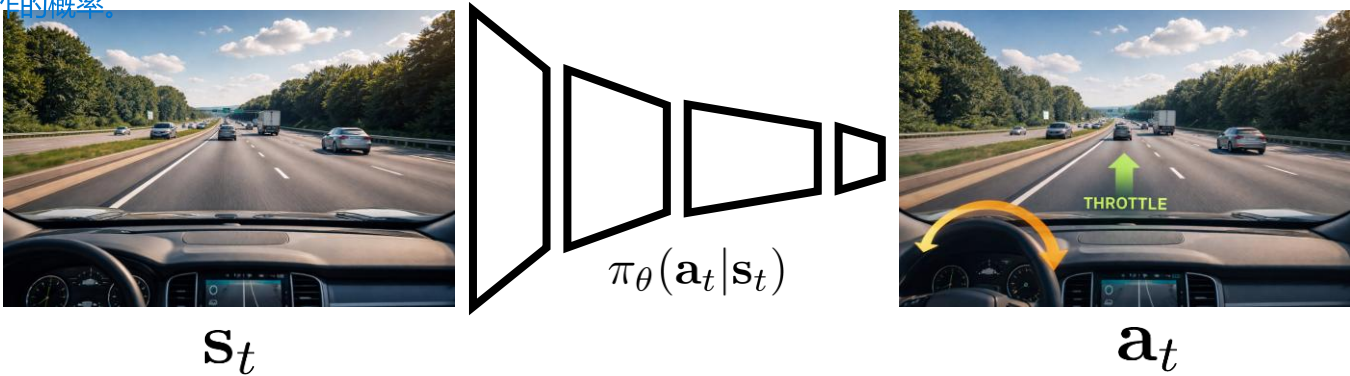
policy gradient:
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right)$$

maximum likelihood:
$$\nabla_{\theta} J_{\text{ML}}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right)$$

没有奖励相关的项，它只关注“让智能体更可能采样到已经发生过的轨迹”

策略梯度的逻辑
如果某条轨迹的累积奖励很高（比如安全到达目的地），会放大这条轨迹的动作概率梯度，让策略更倾向于学习这类高奖励轨迹的动作选择逻辑；反之，低奖励甚至负奖励（比如发生碰撞）的轨迹，会抑制对应动作的概率。

最大似然估计的逻辑
它只会关注“让智能体更可能重复采样到当前轨迹”，比如如果采样到的轨迹是“一直保持当前车速行驶”，最大似然估计会让策略更倾向于选择这个动作，但它不会判断这个动作是否安全、是否能高效到达目的地，哪怕这个动作最终导致了碰撞，只要它在采样轨迹中出现过，就会被强化。



Example: Gaussian policies

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right)$$

example: $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) = \mathcal{N}(f_{\text{neural network}}(\mathbf{s}_t); \Sigma)$

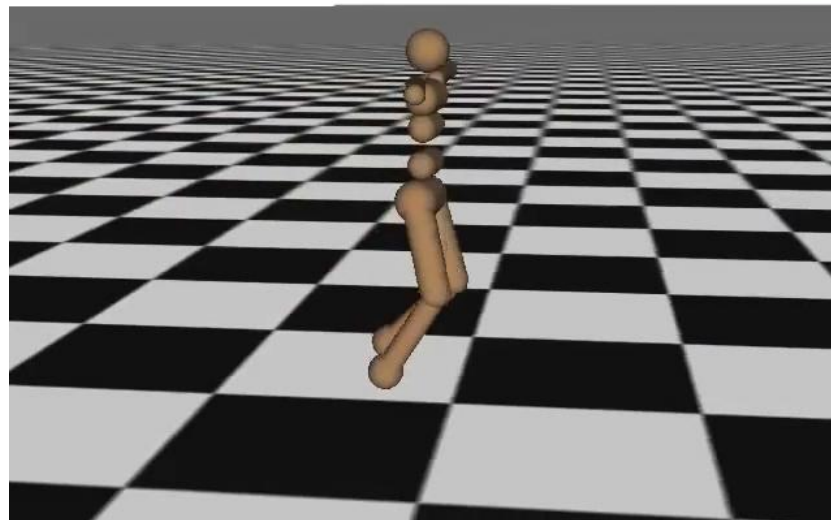
$$\log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) = -\frac{1}{2} \|f(\mathbf{s}_t) - \mathbf{a}_t\|_{\Sigma}^2 + \text{const}$$

$$\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) = -\frac{1}{2} \Sigma^{-1} (f(\mathbf{s}_t) - \mathbf{a}_t) \frac{df}{d\theta}$$

REINFORCE algorithm:

1. sample $\{\tau^{(i)}\}$ from $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ (run the policy)
2. $\nabla_{\theta} J(\theta) \approx \sum_i \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) \left(\sum_t r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right)$
3. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$

Iteration 2000



高斯策略是一种常用的连续动作空间策略
在给定状态下，策略输出一个高斯分布的动作，让机器人在行走时既能保持稳定的步态，又能有一定的探索性，最终学会在棋盘格地面上稳定行走。

What did we just do?

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right)$$

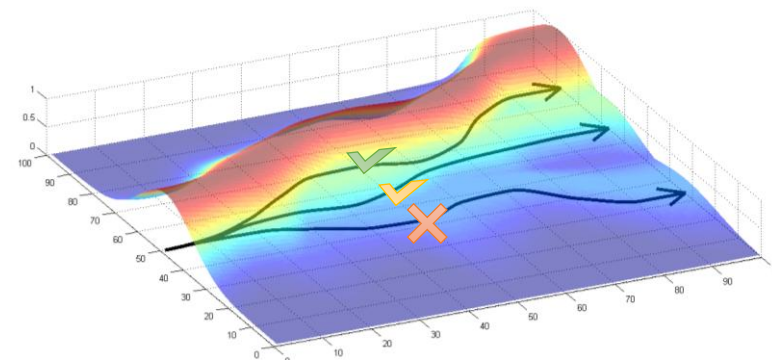
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \underbrace{\nabla_{\theta} \log \pi_{\theta}(\tau^{(i)})}_{\sum_{t=1}^H \nabla_{\theta} \log_{\theta} \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})} r(\tau^{(i)})$$

maximum likelihood: $\nabla_{\theta} J_{\text{ML}}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(\tau^{(i)})$

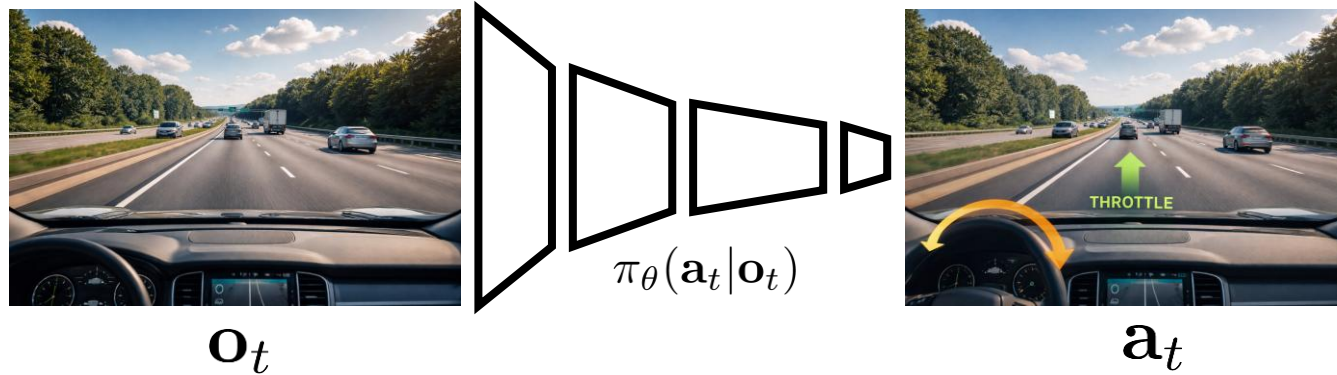
good stuff is made more likely

bad stuff is made less likely

simply formalizes the notion of “trial and error”!



Partial observability



$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{o}_t^{(i)}) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) \right)$$

Markov property is not actually used!

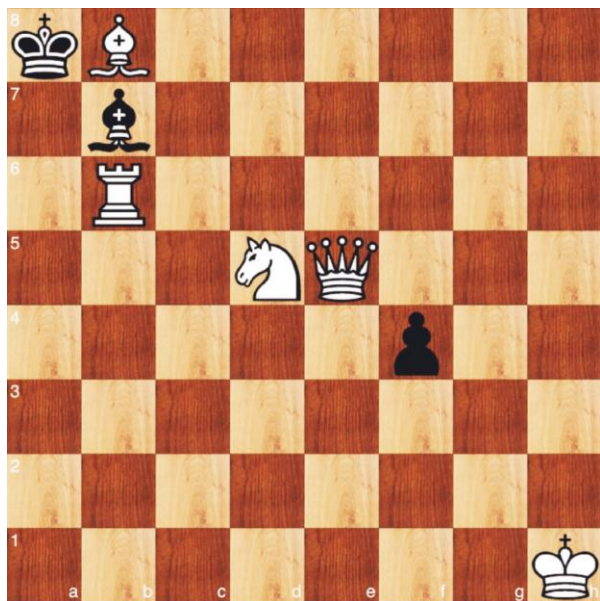
Can use policy gradient in partially observed MDPs without modification

What is wrong with the policy gradient?

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(\tau) r(\tau)$$

What we want: positive multipliers on *good* moves,
negative multipliers on *bad* moves

Example: preset position chess



R = +1 for winning, -1 for losing

What we get:

positive multipliers on *lucky* starts,
negative multipliers on *unlucky* starts

negative multipliers on good moves
when you made a mistake later

positive multipliers on bad moves when
your opponent randomly made a
mistake later

these issues “average out”
with enough samples, but
we might need a very large
number of samples to get
there

“high variance”

Part 3:

Variance reduction

What is wrong with the policy gradient?

参考策略theta的目标函数梯度

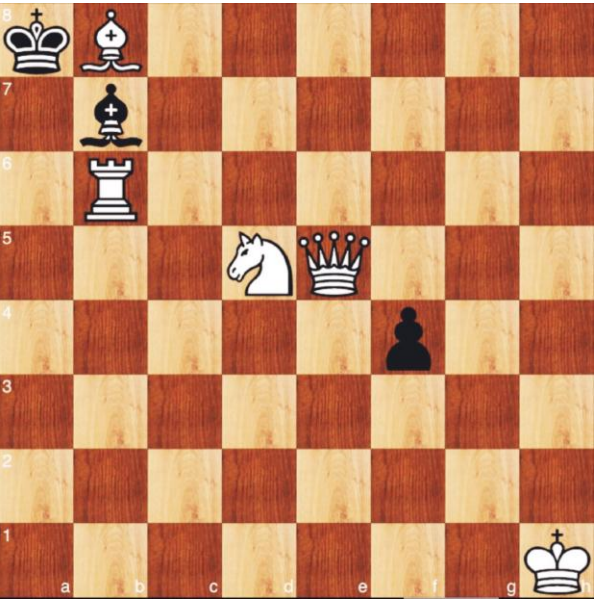
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(\tau) r(\tau)$$

$\pi_{\theta}(\tau)$ 是在当前策略theta下生成轨迹tau的概率
 $r(\tau)$ 是整个轨迹tau的奖励函数

What we want: positive multipliers on *good* moves,
negative multipliers on *bad* moves

我们希望策略梯度能实现：对好的动作给出正向的梯度更新（正乘数），对坏的动作给出负向的梯度更新（负乘数）

Example: preset position chess



What we get:

策略梯度的缺陷：

开局的幸运与否
对于策略评判的干扰

positive multipliers on *lucky* starts,
negative multipliers on *unlucky* starts

negative multipliers on good moves
when you made a mistake later

positive multipliers on bad moves when
your opponent randomly made a
mistake later

后续动作掩盖当前动作价值，比如后续动作很坏但是当前动作好
但是这仍然被评价为negative的move

these issues “average out”
with enough samples, but
we might need a very large
number of samples to get
there

“high variance”

R = +1 for winning, -1 for losing

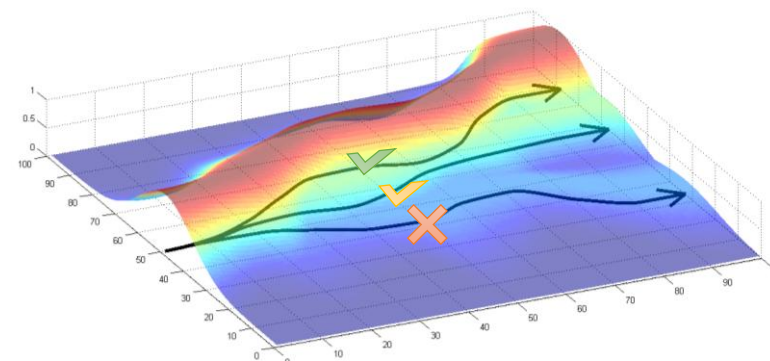
Baselines

a convenient identity

$$p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) = \nabla_{\theta} p_{\theta}(\tau)$$

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log p_{\theta}(\tau) [r(\tau) - b]$$

$$b = \frac{1}{N} \sum_{i=1}^N r(\tau) \quad \text{but... are we } \textit{allowed} \text{ to do that??}$$



$$\mathbb{E}[\nabla_{\theta} \log p_{\theta}(\tau) b] = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) b d\tau = \int \nabla_{\theta} p_{\theta}(\tau) b d\tau = b \nabla_{\theta} \int p_{\theta}(\tau) d\tau = b \nabla_{\theta} 1 = 0$$

通过减去平均奖励，过滤掉和当前轨迹无关的“噪声”奖励，降低梯度估计的方差，让梯度更新更稳定，减少对大量样本的依赖

subtracting a baseline is *unbiased* in expectation!

average reward is *not* the best baseline, but it's pretty good!

Causality

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \right) \left(\sum_{t=1}^H r(\mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)}) - b_t \right)$$

Causality: policy at time t' cannot affect reward at time t when $t < t'$ 因果性, 现有策略不能影响之前时间的奖励

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \left(\underbrace{\sum_{t'=t}^H r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)})}_{\text{"reward to go"} \hat{Q}_t^{(i)}} - b_t \right)$$

Part 4:

Practical implementation

Policy gradient with automatic differentiation

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \overbrace{\left(\sum_{t'=t}^H r(\mathbf{s}_{t'}^{(i)}, \mathbf{a}_{t'}^{(i)}) \right)}^{\hat{Q}_t^{(i)}}$$

传统策略梯度需要显式计算累积奖励的梯度，效率低下
通过构造伪损失函数，将策略梯度转化为该函数的梯度，利用自动微分工具可以高效计算

pretty inefficient to compute these explicitly!

How can we compute policy gradients with automatic differentiation?

We need a graph such that its gradient is the policy gradient!

maximum likelihood: $\nabla_{\theta} J_{\text{ML}}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})$ $J_{\text{ML}}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)})$

Just implement “pseudo-loss” as a weighted maximum likelihood:

$$\tilde{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \hat{Q}_t^{(i)}$$

cross entropy (discrete) or squared error (Gaussian)

Policy gradient with automatic differentiation

Pseudocode example (with discrete actions):

Maximum likelihood:

Given:

actions - (N*T) x Da tensor of actions

states - (N*T) x Ds tensor of states

Build the graph:

logits = policy.predictions(states) # This should return (N*T) x Da tensor of action logits

negative_likelihoods = tf.nn.softmax_cross_entropy_with_logits(labels=actions, logits=logits)

loss = tf.reduce_mean(negative_likelihoods)

gradients = loss.gradients(loss, variables)

logits是未经过softmax等激活函数处理的原始分数，通过策略网络的predictions函数，以状态states作为输入，输出logits

使用TensorFlow中的函数计算负对数似然。这里将真实动作actions作为标签，网络输出的logits作为输入，计算两者之间的交叉熵，衡量预测和真实动作之间的差异。

Policy gradient with automatic differentiation

Pseudocode example (with discrete actions):

Policy gradient:

```
# Given:
# actions - (N*T) x Da tensor of actions
# states - (N*T) x Ds tensor of states
# q_values - (N*T) x 1 tensor of estimated state-action values (with baseline subtracted)
# Build the graph:
logits = policy.predictions(states) # This should return (N*T) x Da tensor of action logits
negative_likelihoods = tf.nn.softmax_cross_entropy_with_logits(labels=actions, logits=logits)
weighted_negative_likelihoods = tf.multiply(negative_likelihoods, q_values)
loss = tf.reduce_mean(weighted_negative_likelihoods)
gradients = loss.gradients(loss, variables)
```

$$\tilde{J}(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \log \pi_{\theta}(\mathbf{a}_t^{(i)} | \mathbf{s}_t^{(i)}) \hat{Q}_t^{(i)}$$

q_values

Policy gradient in practice

- Remember that the gradient has high variance
 - This isn't the same as supervised learning!
 - Gradients will be really noisy!
- Consider using much larger batches
- Tweaking learning rates is very hard
 - Adaptive step size rules like ADAM can be OK-ish
 - We'll learn about policy gradient-specific learning rate adjustment methods later!